

Sprint / Module 23: Capstone and Final Assessment — Full Structured Note

1. Current Progress Analysis

You are now at the final stage of the linked-list curriculum.

Before this module, you already studied almost every major linked-list topic:

Area	Status
Basic pointer mechanics	Covered
Singly linked list operations	Covered
Circular singly linked lists	Covered
Doubly linked lists	Covered
Circular doubly linked lists	Covered
Choosing the right structure	Covered
Full integration and final testing	Current module

The important change in Module 23 is this:

Modules 1 to 22 taught you separate linked-list skills.

Module 23 checks whether you can combine those skills into complete working implementations.

In earlier modules, you learned one operation at a time. For example:

```
Insert();  
Delete();  
Display();  
Search();
```

In Module 23, you must show that you can use all of these together without:

- losing the head pointer,
- breaking the list chain,
- creating incorrect circular links,
- forgetting `prev` links in doubly linked lists,
- missing edge cases,

- or giving wrong complexity.

The main goal is not only to memorize code. The goal is to understand what each pointer is doing.

2. Big Idea of Module 23

Module 23 is a capstone module.

A capstone means a final combined project or final check.

This module does not mainly teach a new linked-list type. Instead, it asks you to prove that you can use everything you already learned.

Simple definition:

Module 23 is about building, testing, tracing, and explaining linked-list operations as a complete system.

A beginner-friendly rule:

You have not mastered linked lists just because you can copy code.

You have mastered linked lists when you can draw the list, explain the pointers, update links safely, dry-run the operation, and state the complexity.

Every operation should be understood in four layers:

1. Pointer meaning
 2. Code
 3. Dry-run trace
 4. Complexity
-

3. The Four-Layer Learning Method

For every linked-list operation, use this method.

Layer 1: Pointer Meaning

Before writing code, answer:

- What does `first` or `Head` point to?
- What does `p` point to?
- What does `q` point to?
- What does `r` point to?

- What does `t` point to?
- Which pointer is moving?
- Which pointer must stay safe?

Example:

```
first -> 10 -> 20 -> 30 -> NULL
      q      p
```

Here:

- `first` points to the first node.
- `q` points to the node before `p`.
- `p` points to the current node.

Layer 2: Code

After you know the pointer roles, write the code.

Example deletion line:

```
q->next = p->next;
```

Meaning:

Make `q` skip over `p`.

Layer 3: Dry Run

A dry run means tracing the code by hand.

You should show how pointers move and how the list changes after every important line.

Example dry-run table:

Step	p	q	r	first / Head	List shape
Start	10	NULL	NULL	10	10 -> 20 -> 30 -> NULL

Layer 4: Complexity

After the operation works, explain the time and space cost.

Examples:

- If the function visits every node, time is usually $O(n)$.
- If the function only changes a fixed number of links, time is usually $O(1)$.
- If the function uses recursion, stack space is usually $O(n)$.
- If the function uses only pointer variables, extra space is usually $O(1)$.

Part A: C Singly Linked List Capstone

4. What Is a Singly Linked List?

A singly linked list is a list where every node points only forward.

Example:

```
first -> 10 -> 20 -> 30 -> NULL
```

Each node has:

```
[data | next]
```

The `next` field stores the address of the next node.

C node structure:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Meaning:

Field	Meaning
<code>data</code>	Stores the value
<code>next</code>	Stores the address of the next node

The last node points to `NULL`.

5. Singly Linked List Functions to Implement

In the capstone, you should be able to implement these singly linked-list operations in C:

```
create
Display
count
sum
max
min
Search
MoveToHeadSearch
Insert
InsertLast
SortedInsert
isSorted
RemoveDuplicates
Delete
Reverse
Concatenate
Merge
isLoop
```

These functions test the main skills from the earlier modules.

6. Important Singly Linked List Pointer Rules

Rule 1: Move forward using `p = p->next`

```
p = p->next;
```

Meaning:

Move pointer `p` to the next node.

Example:

```
p
|
v
10 -> 20 -> 30 -> NULL
```

After:

```
p = p->next;
```

Now:

```
    p
    |
    v
10 -> 20 -> 30 -> NULL
```

Rule 2: Do not move `first` carelessly

`first` is the outside pointer to the first node.

Example:

```
first -> 10 -> 20 -> 30 -> NULL
```

If you do this:

```
first = first->next;
```

then `first` moves to `20`.

Now the node `10` may be lost if no other pointer stores its address.

Correct traversal pattern:

```
struct Node *p = first;

while (p != NULL) {
    printf("%d ", p->data);
    p = p->next;
}
```

Use `p` to move. Keep `first` safe.

Rule 3: Head insertion uses two link changes

To insert a node at the beginning:

```
t->next = first;  
first = t;
```

Meaning:

1. Make the new node point to the old first node.
2. Move `first` to the new node.

Example:

Before:

```
first -> 20 -> 30 -> NULL
```

New node:

```
t -> 10
```

After:

```
first -> 10 -> 20 -> 30 -> NULL
```

The order matters. If you move `first` first, you may lose the old list.

Rule 4: Deletion skips a node

To delete a middle node in a singly linked list:

```
q->next = p->next;  
free(p);
```

Meaning:

- `p` points to the node to delete.
- `q` points to the node before `p`.
- `q->next = p->next` makes `q` skip `p`.

- `free(p)` releases the deleted node.

Example:

Before deleting `30`:

```
10 -> 20 -> 30 -> 40 -> NULL
      q      p
```

After:

```
10 -> 20 -> 40 -> NULL
```

7. Singly Linked List Display

Display means printing all nodes.

Code:

```
void Display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}
```

Explanation:

Code	Meaning
<code>while (p != NULL)</code>	Continue while <code>p</code> points to a real node
<code>printf("%d ", p->data)</code>	Print the current node data
<code>p = p->next</code>	Move to the next node

Complexity:

Type	Complexity
Time	<code>O(n)</code>
Extra space	<code>O(1)</code>

Why time is $O(n)$:

The function visits every node once.

8. Count, Sum, Max, and Min

These are traversal-based operations.

They all follow the same basic pattern:

```
while (p != NULL) {  
    // use p->data  
    p = p->next;  
}
```

Count

Count finds how many nodes are in the list.

```
int count(struct Node *p) {  
    int c = 0;  
    while (p != NULL) {  
        c++;  
        p = p->next;  
    }  
    return c;  
}
```

Sum

Sum adds all values.

```
int sum(struct Node *p) {  
    int total = 0;  
    while (p != NULL) {  
        total += p->data;  
        p = p->next;  
    }  
    return total;  
}
```

Max

Max finds the largest value.

```
int max(struct Node *p) {
    int m = INT32_MIN;
    while (p != NULL) {
        if (p->data > m) {
            m = p->data;
        }
        p = p->next;
    }
    return m;
}
```

Min

Min finds the smallest value.

```
int min(struct Node *p) {
    int m = INT32_MAX;
    while (p != NULL) {
        if (p->data < m) {
            m = p->data;
        }
        p = p->next;
    }
    return m;
}
```

Complexity for all four:

Operation	Time	Extra space
Count	$O(n)$	$O(1)$
Sum	$O(n)$	$O(1)$
Max	$O(n)$	$O(1)$
Min	$O(n)$	$O(1)$

9. Search in a Singly Linked List

Search means finding a node that contains a given value.

The value we search for is called the key.

Example:

List: 8 -> 3 -> 7 -> 12 -> NULL
Key: 12

Search checks:

8? no
3? no
7? no
12? yes

Code:

```
struct Node *Search(struct Node *p, int key) {  
    while (p != NULL) {  
        if (p->data == key) {  
            return p;  
        }  
        p = p->next;  
    }  
    return NULL;  
}
```

Return values:

Situation	Return
Key found	Address of found node
Key not found	NULL

Complexity:

Case	Time
Key at first node	0(1)

Case	Time
Key at last node	$O(n)$
Key not found	$O(n)$

Extra space: $O(1)$.

10. Move-to-Head Search

Move-to-head search improves future searches.

If a key is found, that node is moved to the front of the list.

Example:

Before searching 30:

```
first -> 10 -> 20 -> 30 -> 40 -> NULL
```

After finding 30 and moving it to head:

```
first -> 30 -> 10 -> 20 -> 40 -> NULL
```

Main link changes:

```
q->next = p->next;
p->next = first;
first = p;
```

Meaning:

1. Remove `p` from its old position.
2. Make `p` point to the old first node.
3. Make `first` point to `p`.

Important condition:

Move-to-head needs both `p` and `q`.

- `p` points to the found node.
- `q` points to the node before `p`.

If the found node is already the first node, there is no need to move it.

11. Insert in a Singly Linked List

Insertion means creating a new node and connecting it into the list.

Insertion Index Convention

For insertion:

Index	Meaning
0	Insert before the first node
1	Insert after the first node
2	Insert after the second node
n	Insert after the last node

If the list has `n` nodes, valid insertion indexes are:

0 to n

Invalid indexes:

-1, n + 1, 100

Insert at Head

```
t->next = first;  
first = t;
```

Insert After a Position

```
t->next = p->next;  
p->next = t;
```

Correct order:

1. `t->next = p->next;`
2. `p->next = t;`

Why this order is important:

If you do `p->next = t` first, you may lose the node that originally came after `p`.

Complexity:

Case	Time	Extra space
Insert at head	<code>O(1)</code>	<code>O(1)</code>
Insert at position	<code>O(n)</code>	<code>O(1)</code>

12. InsertLast Optimization

InsertLast means inserting at the end of the list.

If we do not have a `last` pointer, inserting at the end may require traversal every time.

That can make repeated end insertions slow.

With a `last` pointer:

```
last->next = t;  
last = t;
```

Meaning:

1. Attach the new node after the current last node.
2. Move `last` to the new final node.

If the list is empty:

```
first = last = t;
```

Complexity:

Method	Time per append
Without <code>last</code> pointer	<code>O(n)</code>
With <code>last</code> pointer	<code>O(1)</code>

13. SortedInsert

SortedInsert inserts a value into the correct place in a sorted list.

Example:

```
3 -> 7 -> 9 -> 15 -> 20 -> NULL
```

Insert 18:

```
3 -> 7 -> 9 -> 15 -> 18 -> 20 -> NULL
```

Sorted insertion uses value order, not index.

Main loop:

```
while (p != NULL && p->data < x) {  
    q = p;  
    p = p->next;  
}
```

Why `p != NULL` comes first:

This is safe:

```
p != NULL && p->data < x
```

This is unsafe:

```
p->data < x && p != NULL
```

Because if `p` is `NULL`, then `p->data` is invalid.

Insertion cases:

Case	Action
Empty list	New node becomes <code>first</code>
Insert before first	<code>t->next = first; first = t;</code>

Case	Action
Insert middle/end	<code>t->next = q->next; q->next = t;</code>

Complexity:

Type	Complexity
Time	<code>O(n)</code>
Extra space	<code>O(1)</code>

14. isSorted

`isSorted` checks whether a list is in increasing order.

Example sorted list:

```
3 -> 5 -> 8 -> 12 -> NULL
```

Example unsorted list:

```
3 -> 5 -> 2 -> 12 -> NULL
```

The list is not sorted because `2` comes after `5`.

Basic idea:

- Keep track of the previous value.
- If the current value is smaller than the previous value, the list is not sorted.

Complexity:

Type	Complexity
Time	<code>O(n)</code>
Extra space	<code>O(1)</code>

15. RemoveDuplicates

`RemoveDuplicates` removes repeated values from a sorted list.

Important condition:

This method assumes the list is already sorted.

Example:

```
3 -> 5 -> 5 -> 7 -> 7 -> 7 -> 10 -> NULL
```

After removing duplicates:

```
3 -> 5 -> 7 -> 10 -> NULL
```

Because the list is sorted, duplicates appear next to each other.

Typical pointer roles:

Pointer	Role
<code>p</code>	Current node
<code>q</code>	Node after <code>p</code> , possibly duplicate

If values are equal:

```
p->next = q->next;  
free(q);  
q = p->next;
```

If values are different:

```
p = q;  
q = q->next;
```

Complexity:

Type	Complexity
Time	<code>O(n)</code>
Extra space	<code>O(1)</code>

16. Delete in a Singly Linked List

Deletion means removing a node from the chain and freeing its memory.

Deletion has two parts:

1. Change links so the list skips the deleted node.
2. Free the deleted node.

Deletion Index Convention

For deletion:

Index	Meaning
1	Delete the first node
2	Delete the second node
3	Delete the third node
n	Delete the last node

Valid deletion indexes:

1 to n

Invalid indexes:

0, -1, n + 1, 100

Important:

In insertion, index 0 is valid.

In deletion, index 0 is invalid.

Delete First Node

```
q = first;
x = q->data;
first = first->next;
free(q);
return x;
```

Why save `q`?

Because after moving `first`, we still need the old first node so we can free it.

Delete Middle or Last Node

```
q->next = p->next;  
x = p->data;  
free(p);  
return x;
```

Complexity:

Case	Time	Extra space
Delete first node	<code>O(1)</code>	<code>O(1)</code>
Delete by index	<code>O(n)</code>	<code>O(1)</code>

17. Reverse a Singly Linked List

Reversing means changing the actual links so the list order becomes opposite.

Original:

```
first -> 10 -> 20 -> 30 -> NULL
```

After reversal:

```
first -> 30 -> 20 -> 10 -> NULL
```

Important:

Reverse display is not real reversal.
Real reversal changes the links.

Sliding-Pointer Reversal

Use three pointers:

Pointer	Meaning
p	Current node in the original list
q	Current node in reversed part
r	Node before q in reversed part

Core pattern:

```
r = q;
q = p;
p = p->next;
q->next = r;
```

Important beginner rule:

Move p forward before changing q->next.
Otherwise, you may lose the rest of the list.

Full idea:

```
while (p != NULL) {
    r = q;
    q = p;
    p = p->next;
    q->next = r;
}
first = q;
```

18. Dry Run: Reverse Singly Linked List

Given:

```
10 -> 20 -> 30 -> NULL
```

Initial state:

Step	p	q	r	List
Start	10	NULL	NULL	10 -> 20 -> 30 -> NULL

First loop

Code:

```
r = q;      // r = NULL
q = p;      // q = 10
p = p->next; // p = 20
q->next = r; // 10 -> NULL
```

State:

Step	p	q	r	Reversed part	Remaining part
1	20	10	NULL	10 -> NULL	20 -> 30 -> NULL

Second loop

State:

Step	p	q	r	Reversed part	Remaining part
2	30	20	10	20 -> 10 -> NULL	30 -> NULL

Third loop

State:

Step	p	q	r	Reversed part	Remaining part
3	NULL	30	20	30 -> 20 -> 10 -> NULL	empty

Finally:

```
first = q;
```

Final list:

```
30 -> 20 -> 10 -> NULL
```

Complexity:

Type	Complexity
Time	<code>O(n)</code>
Extra space	<code>O(1)</code>

19. Concatenate Two Singly Linked Lists

Concatenation means attaching the second list after the first list.

Example:

```
first  -> 2 -> 9 -> 6 -> NULL
second -> 10 -> 15 -> NULL
```

After concatenation:

```
first -> 2 -> 9 -> 6 -> 10 -> 15 -> NULL
```

Main idea:

Find the last node of the first list and make it point to the first node of the second list.

Correct loop:

```
while (p->next != NULL) {
    p = p->next;
}
```

Why this loop?

It stops on the last node.

Wrong loop:

```
while (p != NULL) {
    p = p->next;
}
```

This is wrong for concatenation because it stops after the last node, where `p == NULL`.

Then this would be invalid:

```
p->next = second;
```

Main connection:

```
p->next = second;  
second = NULL;
```

Complexity:

Type	Complexity
Time	$O(n)$ for first list
Extra space	$O(1)$

If we already have a `last` pointer, concatenation can be $O(1)$.

20. Merge Two Sorted Lists

Merge means combining two sorted lists into one sorted list.

Example:

```
first  -> 2 -> 8 -> 10 -> NULL  
second -> 4 -> 7 -> NULL
```

After merge:

```
third -> 2 -> 4 -> 7 -> 8 -> 10 -> NULL
```

Important rule:

Merge assumes both input lists are already sorted.

Merge does not need to create new nodes. It can rearrange existing links.

Pointer roles:

Pointer	Meaning
<code>first</code>	First sorted list
<code>second</code>	Second sorted list
<code>third</code>	Start of merged list
<code>last</code>	Last node in merged list

Basic idea:

1. Compare the first node of each list.
2. Attach the smaller node to the merged list.
3. Move forward in the list where the smaller node came from.
4. Repeat until one list becomes empty.
5. Attach the remaining list.

Complexity:

Type	Complexity
Time	$O(m + n)$
Extra space	$O(1)$

Where:

- `m` is the number of nodes in the first list.
- `n` is the number of nodes in the second list.

21. Loop Detection in a Singly Linked List

A normal singly linked list ends at `NULL`.

Example:

```
10 -> 20 -> 30 -> NULL
```

A looping list does not end normally.

Example:


```

10 -> 20 -> 30 -> 40
      ^           |
      |_____|

```

Here, 40 points back to 20.

Normal display may run forever because p may never become NULL.

Floyd's Cycle Detection

Floyd's cycle detection uses two pointers:

Pointer	Speed	Meaning
p	Moves 1 step	Slow pointer
q	Moves 2 steps	Fast pointer

Logic:

- If the list has no loop, the fast pointer reaches NULL.
- If the list has a loop, the fast pointer eventually meets the slow pointer.

Condition for loop found:

```
p == q
```

Why do-while is used:

At the start:

```
p = q = first;
```

So p == q is already true before movement.

We need both pointers to move first, then check if they meet.

Complexity:

Type	Complexity
Time	O(n)
Extra space	O(1)

Part B: C Circular Singly Linked List Capstone

22. What Is a Circular Singly Linked List?

A circular singly linked list is like a singly linked list, but the last node points back to the first node.

Normal singly linked list:

```
10 -> 20 -> 30 -> NULL
```

Circular singly linked list:

```
Head -> 10 -> 20 -> 30
      ^           |
      |           |
      |_____||
```

The most important rule:

```
last->next = Head;
```

There is no normal `NULL` ending.

23. Circular Singly Linked List Functions to Implement

In the capstone, you should implement:

```
create
Display
RDisplay
Length
Insert
Delete
```

These functions test whether you understand circular traversal and circular link maintenance.

24. Important Circular Singly Cases

Case	Condition
Empty list	Head == NULL
One-node circular list	Head->next == Head
Multi-node circular list	Last node points back to Head

Empty circular list

```
Head -> NULL
```

One-node circular list

```
Head -> 10
      ^|
      ||
      |_  
      |
```

Code condition:

```
Head->next = Head;
```

Multi-node circular list

```
Head -> 10 -> 20 -> 30
      ^           |
      |_____||
```

Code condition:

```
last->next = Head;
```

25. Circular Display

A normal list display uses:

```
while (p != NULL)
```

This is wrong for circular lists because a circular list may never reach `NULL`.

Correct circular display:

```
void Display(struct Node *h) {  
    if (h == NULL) {  
        return;  
    }  
  
    struct Node *p = h;  
  
    do {  
        printf("%d ", p->data);  
        p = p->next;  
    } while (p != h);  
}
```

Why `do-while` is needed:

At the start:

```
p == h
```

If we used:

```
while (p != h)
```

then the loop would not run even once.

A `do-while` loop solves this because it:

1. Visits the first node.
2. Moves to the next node.
3. Stops when it returns to `Head`.

Complexity:

Type	Complexity
Time	<code>O(n)</code>

Type	Complexity
Extra space	0(1)

26. Circular Length

A circular length function counts nodes until it returns to `Head` .

Code:

```
int Length(struct Node *h) {
    if (h == NULL) {
        return 0;
    }

    int len = 0;
    struct Node *p = h;

    do {
        len++;
        p = p->next;
    } while (p != h);

    return len;
}
```

Important:

Do not use:

```
while (p != NULL)
```

for circular lists.

Complexity:

Type	Complexity
Time	0(n)
Extra space	0(1)

27. Circular Singly Insertion

Insertion means adding a new node while keeping the circle correct.

Insertion index convention:

Index	Meaning
0	Insert before <code>Head</code> ; new node becomes <code>Head</code>
1	Insert after first node
2	Insert after second node
n	Insert after last node

Valid indexes:

0 to n

Important cases

Case	What must happen
Empty list	New node points to itself
Insert at index 0	New node becomes <code>Head</code>
Insert after position	Normal link insertion
Insert at end	New last node points to <code>Head</code>

Empty list insertion

If the list is empty:

```
Head = t;  
Head->next = Head;
```

Meaning:

The new node is the only node, so it points to itself.

Insert before Head in non-empty list

To insert before `Head`, we must find the last node because the last node must point to the new `Head`.

Main idea:

```
t->next = Head;  
last->next = t;  
Head = t;
```

Meaning:

1. New node points to old `Head`.
2. Old last node points to new node.
3. `Head` moves to new node.

28. Circular Singly Deletion

Deletion means removing a node while preserving the circle.

Deletion index convention:

Index	Meaning
1	Delete <code>Head</code>
2	Delete second node
n	Delete last node

Valid indexes:

1 to n

Invalid indexes:

0, -1, n + 1, 100

Important circular deletion cases

Case	What happens
Empty list	Return failure
One-node list	Free node and set <code>Head = NULL</code>
Delete <code>Head</code> in multi-node list	Last node must point to new <code>Head</code>

Case	What happens
Delete middle node	Previous node skips deleted node
Delete last node	Previous node points back to <code>Head</code>

One-node deletion

If:

```
Head->next == Head
```

then there is only one node.

After deletion:

```
free(Head);
Head = NULL;
```

Delete Head in multi-node circular list

Main idea:

1. Find the last node.
2. Save old `Head`.
3. Move `Head` to the next node.
4. Make last node point to new `Head`.
5. Free old `Head`.

Conceptual code:

```
p = Head;
while (p->next != Head) {
    p = p->next;
}

q = Head;
Head = Head->next;
p->next = Head;
free(q);
```


Part C: C Doubly Linked List Capstone

29. What Is a Doubly Linked List?

A doubly linked list is a list where each node has two links:

```
[prev | data | next]
```

Example:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Each node can move:

- forward using `next`,
- backward using `prev`.

C node structure:

```
struct Node {  
    struct Node *prev;  
    int data;  
    struct Node *next;  
};
```

30. Doubly Linked List Functions to Implement

In the capstone, you should implement:

```
create  
display  
length  
Insert  
Delete  
reverse
```

31. Doubly Linked List Invariants

An invariant is a rule that must stay true if the list is correct.

Main doubly linked-list invariant:

```
A->next == B  
B->prev == A
```

Meaning:

If node **A** points forward to node **B**, then node **B** must point backward to node **A**.

Example:

```
10 <-> 20
```

Correct links:

```
10->next == 20  
20->prev == 10
```

Other important rules:

Node	Required link
First node	<code>prev == NULL</code>
Last node	<code>next == NULL</code>
One-node list	<code>prev == NULL</code> and <code>next == NULL</code>

32. Doubly Linked List Creation

When creating a doubly linked list from an array, the first node is special.

First node:

```
first = malloc(sizeof(struct Node));  
first->prev = NULL;  
first->data = A[0];
```

```
first->next = NULL;
last = first;
```

Remaining nodes:

```
t = malloc(sizeof(struct Node));
t->prev = last;
t->data = A[i];
t->next = NULL;
last->next = t;
last = t;
```

The important extra line compared with singly linked lists is:

```
t->prev = last;
```

This creates the backward link.

Complexity:

Type	Complexity
Time	$O(n)$
Extra space	$O(n)$ for the nodes

33. Doubly Linked List Display and Length

Display is similar to singly linked-list display.

```
void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}
```

Length:

```
int length(struct Node *p) {
    int len = 0;
```

```

while (p != NULL) {
    len++;
    p = p->next;
}
return len;
}

```

Complexity:

Operation	Time	Extra space
Display	$O(n)$	$O(1)$
Length	$O(n)$	$O(1)$

34. Doubly Linked List Insertion

Insertion in a doubly linked list must update both `next` and `prev` links.

Insertion index convention:

Index	Meaning
0	Insert before first node
1	Insert after first node
2	Insert after second node
n	Insert after last node

Valid indexes:

0 to n

Insert at Head

```

t->prev = NULL;
t->next = first;

if (first != NULL) {
    first->prev = t;
}

```

```
first = t;
```

Why check `first != NULL`?

Because if the list is empty, then `first` is `NULL`, and this is invalid:

```
first->prev = t;
```

Insert in Middle

Suppose we insert `15` between `10` and `20`.

Before:

```
10 <-> 20  
p
```

After:

```
10 <-> 15 <-> 20
```

Main code:

```
t->prev = p;  
t->next = p->next;  
p->next->prev = t;  
p->next = t;
```

Four links are involved:

Code	Meaning
<code>t->prev = p</code>	New node points backward to <code>p</code>
<code>t->next = p->next</code>	New node points forward to old next node
<code>p->next->prev = t</code>	Old next node points backward to new node
<code>p->next = t</code>	<code>p</code> points forward to new node

Insert at End

At the end of the list, this line may be unsafe:

```
p->next->prev = t;
```

Why?

Because if `p` is the last node, then:

```
p->next == NULL
```

So this would mean:

```
NULL->prev = t
```

That is invalid.

Safe version:

```
t->prev = p;
t->next = p->next;

if (p->next != NULL) {
    p->next->prev = t;
}

p->next = t;
```

Complexity:

Case	Time	Extra space
Insert at head	<code>O(1)</code>	<code>O(1)</code>
Insert by index	<code>O(n)</code>	<code>O(1)</code>

35. Doubly Linked List Deletion

Deletion removes a node and preserves both forward and backward links.

Deletion index convention:

Index	Meaning
1	Delete first node
2	Delete second node
n	Delete last node

Valid indexes:

1 to n

Invalid indexes:

0, -1, n + 1, 100

Delete First Node

```
p = first;
x = p->data;
first = first->next;

if (first != NULL) {
    first->prev = NULL;
}

free(p);
return x;
```

Why check `first != NULL`?

If the list had only one node, then after:

```
first = first->next;
```

`first` becomes `NULL`.

So this would be unsafe:

```
first->prev = NULL;
```

Delete Middle Node

If `p` points to the node being deleted:

```
p->prev->next = p->next;  
p->next->prev = p->prev;  
free(p);
```

Delete Last Node

For the last node:

```
p->next == NULL
```

So this is unsafe:

```
p->next->prev = p->prev;
```

Safe deletion pattern:

```
p->prev->next = p->next;  
  
if (p->next != NULL) {  
    p->next->prev = p->prev;  
}  
  
free(p);
```

Complexity:

Case	Time	Extra space
Delete first	<code>O(1)</code>	<code>O(1)</code>
Delete by index	<code>O(n)</code>	<code>O(1)</code>

36. Doubly Linked List Reversal

To reverse a doubly linked list, swap each node's `prev` and `next` links.

Original:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

After reversal:

```
NULL <- 30 <-> 20 <-> 10 -> NULL
```

Main idea:

For every node:

```
temp = p->next;
p->next = p->prev;
p->prev = temp;
p = p->prev;
```

Why move using `p = p->prev` after swapping?

Because after the swap, the old `next` link is now stored in `prev`.

At the end, update `first` to the new first node.

Complexity:

Type	Complexity
Time	<code>O(n)</code>
Extra space	<code>O(1)</code>

Part D: Circular Doubly Linked List Prototype

37. What Is a Circular Doubly Linked List?

A circular doubly linked list combines two ideas:

1. Circular structure
2. Two-way movement

Each node has:

```
[prev | data | next]
```

The list is circular:

```
Head -> 10 <-> 20 <-> 30
      ^           |
      |           |
      +-----+
```

Important rules:

```
last->next == Head;
Head->prev == last;
```

Meaning:

- From the last node, moving forward returns to `Head`.
- From `Head`, moving backward reaches the last node.

38. Circular Doubly Linked List Cases

Case	Condition
Empty list	<code>Head == NULL</code>
One-node list	<code>Head->next == Head</code> and <code>Head->prev == Head</code>
Multi-node list	Last node links forward to <code>Head</code> , and <code>Head</code> links backward to last

One-node circular doubly list

```
Head -> 10
      ^|
      ||
      |_  
      |
```

Code:

```
Head->next = Head;  
Head->prev = Head;
```

This is different from a normal one-node doubly linked list.

Normal one-node doubly list:

```
first->prev = NULL;  
first->next = NULL;
```

Circular one-node doubly list:

```
Head->prev = Head;  
Head->next = Head;
```

39. Circular Doubly Display

Normal `while (p != NULL)` traversal is wrong because the list does not normally reach `NULL`.

Use `do-while`:

```
void Display(struct Node *h) {  
    if (h == NULL) {  
        return;  
    }  
  
    struct Node *p = h;  
  
    do {  
        printf("%d ", p->data);  
        p = p->next;  
    } while (p != h);  
}
```

```
    } while (p != h);  
}
```

Complexity:

Type	Complexity
Time	$O(n)$
Extra space	$O(1)$

40. Insert Before Head in Circular Doubly List

This is an important prototype operation.

Goal:

Insert a new node before `Head`.

Because the list is circular doubly linked, we can get the last node using:

```
Head->prev
```

This makes insertion before `Head` efficient.

Let:

```
last = Head->prev;
```

Main link updates:

```
t->next = Head;  
t->prev = Head->prev;  
Head->prev->next = t;  
Head->prev = t;  
Head = t;
```

Meaning:

Code	Meaning
<code>t->next = Head</code>	New node points forward to old Head
<code>t->prev = Head->prev</code>	New node points backward to old last node
<code>Head->prev->next = t</code>	Old last node points forward to new node
<code>Head->prev = t</code>	Old Head points backward to new node
<code>Head = t</code>	New node becomes Head

Complexity:

Type	Complexity
Time	<code>O(1)</code>
Extra space	<code>O(1)</code>

Why it is `O(1)`:

We do not need to traverse to the last node because `Head->prev` already points to the last node.

Part E: C++ LinkedList Class Capstone

41. Why Use a C++ Class?

In earlier C-style code, the list may use a global pointer:

```
struct Node *first = NULL;
```

Functions may look like this:

```
Display(first);
Insert(first, 2, 50);
Delete(first, 3);
```

In C++, we can place the data and functions inside a class.

Then we write:

```
LinkedList list;  
list.Display();  
list.Insert(2, 50);  
list.Delete(3);
```

Main idea:

In C, functions often receive `first` as an argument.

In C++, the linked-list object owns `first`.

42. Basic C++ Class Structure

```
class Node {  
public:  
    int data;  
    Node *next;  
};  
  
class LinkedList {  
private:  
    Node *first;  
  
public:  
    LinkedList();  
    LinkedList(int A[], int n);  
    ~LinkedList();  
  
    void Display();  
    int Length();  
    void Insert(int index, int x);  
    int Delete(int index);  
};
```

43. Meaning of `private` and `public`

private

Private data cannot be directly accessed from outside the class.

```
private:
    Node *first;
```

This protects `first`.

Outside code cannot safely do:

```
list.first = nullptr;
```

That is good because accidentally changing `first` could lose the whole list.

public

Public functions can be called from outside the class.

```
public:
    void Display();
```

Outside code can do:

```
list.Display();
```

44. Constructor

A constructor runs automatically when an object is created.

Default constructor:

```
LinkedList() {
    first = nullptr;
}
```

Meaning:

The new list starts empty.

Parameterized constructor:

```
LinkedList(int A[], int n)
```

This creates a list from an array.

Example:

```
int A[] = {10, 20, 30};  
LinkedList list(A, 3);
```

Result:

```
10 -> 20 -> 30 -> nullptr
```

45. Destructor

A destructor runs automatically when the object is destroyed.

Its job is to release heap memory.

Example purpose:

```
~LinkedList();
```

A destructor should walk through the list and delete every node.

Why?

Because nodes created using `new` must be released using `delete`.

C and C++ memory rules:

Language	Create memory	Release memory
C	<code>malloc</code>	<code>free</code>
C++	<code>new</code>	<code>delete</code>

Do not mix them.

Wrong:


```
Node *p = new Node;  
free(p); // wrong
```

Correct:

```
Node *p = new Node;  
delete p;
```

Module 23 Testing Requirements

46. What Module 23 Is Really Testing

Module 23 tests four abilities.

47. Ability 1: Can You Implement?

You should be able to write important functions from memory.

Especially:

```
display  
count  
search  
insert  
delete  
reverse  
Length
```

But implementation alone is not enough.

You must also explain the pointer meaning, dry-run the operation, handle edge cases, and state complexity.

48. Ability 2: Can You Trace?

Tracing means following the code step by step.

For every important operation, make a table like this:

Step	p	q	r	first / Head	List shape
------	---	---	---	--------------	------------

Tracing is important because linked-list bugs are often pointer bugs.

You should trace:

- insertion,
- deletion,
- reversal,
- circular insertion,
- circular deletion,
- doubly linked-list insertion,
- doubly linked-list deletion,
- merge,
- loop detection.

49. Ability 3: Can You Handle Edge Cases?

An edge case is a special situation where normal code may fail.

Important edge cases:

Operation	Edge cases
Display	Empty list
Insert	Index 0, end index, invalid index
Delete	Empty list, index 1, last index, invalid index
Search	Found at head, found at tail, not found
Reverse	Empty list, one-node list, multi-node list
SortedInsert	Empty list, insert before first, insert at end
RemoveDuplicates	No duplicates, all duplicates, duplicates at end
Circular delete	One-node circle, delete head, delete last
Doubly linked-list insert	Insert at head, middle, end
Doubly linked-list reverse	Empty list, one node, multiple nodes

A strong final implementation must not crash on these cases.

50. Ability 4: Can You Explain Complexity?

You should know these rules:

Operation type	Complexity
Only changes fixed pointers	Usually $O(1)$
Traverses the list	Usually $O(n)$
Uses recursion over all nodes	$O(n)$ time and $O(n)$ stack space
Uses only temporary pointers	$O(1)$ extra space
Uses an array copy	$O(n)$ extra space

Examples:

```
Display:  $O(n)$ 
Count:  $O(n)$ 
Search worst case:  $O(n)$ 
Insert at head:  $O(1)$ 
Insert at position:  $O(n)$ 
Delete first node:  $O(1)$ 
Delete middle/end by index:  $O(n)$ 
Reverse sliding pointers:  $O(n)$  time,  $O(1)$  space
Recursive display:  $O(n)$  time,  $O(n)$  space
InsertLast with last pointer:  $O(1)$ 
```

Beginner Final Project Workflow

51. Step 1: Draw Before Coding

Before coding, draw the list.

Example:

```
first -> 10 -> 20 -> 30 -> NULL
```

Then mark the pointers:

```
first -> 10 -> 20 -> 30 -> NULL
      q      p
```

Do not start coding until you know what each pointer means.

52. Step 2: Write the Normal Case

First handle the usual situation.

Example: delete a middle node.

Before:

```
10 -> 20 -> 30 -> 40 -> NULL
   q      p
```

Delete 30 :

```
q->next = p->next;
free(p);
```

After:

```
10 -> 20 -> 40 -> NULL
```

53. Step 3: Add Edge Cases

Ask these questions:

```
What if the list is empty?
What if there is only one node?
What if I delete the head?
What if I delete the last node?
What if the index is invalid?
```

Most linked-list crashes happen because an edge case was ignored.

54. Step 4: Add Tests

For each function, write at least:

```
Normal test  
Edge test  
Invalid test
```

Example for `Delete`:

```
Normal: delete index 3 from 10 -> 20 -> 30 -> 40  
Edge: delete index 1  
Edge: delete last index  
Invalid: delete index 0  
Invalid: delete index larger than length
```

55. Step 5: Write Complexity Comments

Above each function, write a comment like this:

```
// Time: O(n), because we may traverse to the index.  
// Extra space: O(1), because we only use p and q.
```

This forces you to understand the function, not just memorize it.

Final Oral Check Questions

56. Question 1: Why should we not move `first` during traversal?

Answer:

Because `first` is the only external pointer to the start of the list. If we move it forward, we may lose access to the original first node and maybe the whole list. We use a temporary pointer like `p` instead.

57. Question 2: Why is this insertion order correct?

```
t->next = p->next;  
p->next = t;
```

Answer:

First, the new node points to the node that used to come after `p`. Then `p` points to the new node. If we change `p->next` first, we may lose access to the rest of the list.

58. Question 3: Why does circular display use `do-while`?

Answer:

Because at the start, `p == Head`. If we used `while (p != Head)`, the loop would not run even once. A `do-while` loop visits the first node first, then stops when it returns to `Head`.

59. Question 4: Why is binary search bad for normal linked lists?

Answer:

Binary search needs fast access to the middle element. Arrays can access `A[mid]` in $O(1)$, but linked lists must walk node by node to reach the middle. That removes the main advantage of binary search.

60. Question 5: What is the difference between singly and doubly linked-list deletion?

Answer:

In a singly linked list, we often need a previous pointer `q` because the node does not know its previous node. In a doubly linked list, each node has `prev`, so we can use `p->prev` and `p->next` to reconnect the neighbors.

Final Written Check

61. Complexity Table

Fill this from memory until you can explain every row.

Operation	Time	Extra space
Display singly list	$O(n)$	$O(1)$
Recursive display	$O(n)$	$O(n)$
Count iterative	$O(n)$	$O(1)$
Search worst case	$O(n)$	$O(1)$
Insert at head	$O(1)$	$O(1)$
Insert at position	$O(n)$	$O(1)$
Delete first	$O(1)$	$O(1)$
Delete middle/end by index	$O(n)$	$O(1)$
Reverse using sliding pointers	$O(n)$	$O(1)$
Merge two sorted lists	$O(m+n)$	$O(1)$
Floyd loop detection	$O(n)$	$O(1)$
Circular display	$O(n)$	$O(1)$
Doubly linked-list reverse	$O(n)$	$O(1)$

Final Design Check

62. Choosing the Correct Structure

You should be able to choose the correct structure for a problem.

Scenario	Best choice	Reason
Need fast <code>A[i]</code> access	Array	Index access is $O(1)$
Unknown number of items, frequent head insertion	Singly linked list	Flexible size, head insert is $O(1)$
Need to move forward and backward	Doubly linked list	Has <code>prev</code> and <code>next</code>
Need repeated circular turn-taking	Circular singly list	Last node returns to first
Need circular movement in both directions	Circular doubly list	Can move next and prev in a circle

Scenario	Best choice	Reason
Need sorted search with binary search	Array	Linked list cannot jump to the middle efficiently

Simple rule:

Need fast index access? Use an array.
 Need flexible insertion and deletion? Use a linked list.
 Need backward movement? Use a doubly linked list.
 Need looping behavior? Use a circular list.

Final Mastery Checklist

63. Module 23 Checklist

You understand Module 23 when you can do all of this:

- [] Draw singly, circular singly, doubly, and circular doubly lists.
- [] Explain first, Head, last, p, q, r, and t.
- [] Write display from memory.
- [] Write count, sum, max, and min from memory.
- [] Write search and move-to-head search.
- [] Write insert and delete with validation.
- [] Reverse a singly linked list using sliding pointers.
- [] Trace circular insertion and deletion.
- [] Trace doubly linked-list insertion and deletion.
- [] Explain why do-while is needed in circular lists.
- [] Explain why recursive functions use stack space.
- [] State time and space complexity for every major operation.
- [] Choose the right structure for a problem.
- [] Explain every pointer assignment in at least one full function.

64. Final Beginner Rule

The most important lesson of Module 23 is:

Linked-list mastery means you can preserve the chain while changing it.

Never focus only on one line of code.

Always ask:

Before this line, where does every pointer point?
After this line, is every node still reachable?
Did I update first or Head if the first node changed?
Did I handle NULL correctly?
Did I handle circular stopping conditions correctly?
Did I update both next and prev in a doubly linked list?
Did I free or delete removed heap nodes?
Did I test empty, one-node, head, tail, and invalid cases?
Can I explain the time and space complexity?

If you can answer these questions, you are ready to finish the linked-list course.